

Perfecto — te lo explico como un **desarrollo narrativo paso a paso**, justificando cada decisión importante. Imagina que estamos en clase y vamos construyendo el programa mientras explico por qué hacemos cada cosa. Incluyo también un **ejemplo numérico** pequeño (4 cajas) para ver cómo evolucionan individuos entre generaciones.

1) Problema y objetivo (en una frase)

Queremos **seleccionar un subconjunto de cajas** de entre 10 posibles de forma que **maximice el precio total** sin superar la capacidad máxima de la grúa ($C = 1000$ kg). Resolveremos esto con un **algoritmo genético**.

2) Representación — ¿qué es un individuo y por qué lo modelamos así?

Decisión: representar cada posible solución (individuo) como un vector binario de longitud n (n = número de cajas).

Justificación:

- Es natural para problemas de selección 0/1 (incluyo o no incluyo cada caja).
- Facilita operaciones biológicas análogas: cruce por bloques o por genes y mutación bit a bit.

Ejemplo de individuo ($n=10$):

$[1, 0, 1, 0, 0, 1, 0, 0, 1, 0]$ → incluimos las cajas 1,3,6,9.

3) Inicialización de la población — ¿cuántos individuos y cómo generarlos?

Decisión: iniciar con una población N de individuos aleatorios pero **válidos** ($\text{peso} \leq C$). Un valor típico: $N = 30 \dots 100$ (p. ej. 50).

Cómo generar un individuo válido (dos opciones): - **Construcción aleatoria controlada:** barajar índices y agregar ítems uno a uno si caben (determinista respecto al orden aleatorio).

- **Generación+reparación:** generar bits aleatorios y si excede C aplicar un mecanismo de reparación (explico abajo).

Justificación:

- Mantener sólo individuos válidos evita tener que diseñar penalizaciones complejas y cumple la exigencia del enunciado “verificar que ningún individuo supere el límite”.
 - Generar directamente soluciones válidas mejora eficiencia de la búsqueda.
-

4) Función de idoneidad (fitness) — ¿qué medimos y por qué?

Decisión: $\text{fitness}(\text{individuo}) = \text{suma de precios de las cajas incluidas}$, siempre que $\text{peso_total} \leq C$. (Si hay inválidos, se reparan; por tanto asumimos válidos).

Justificación:

- Buscamos maximizar el precio bajo la restricción de peso. Si todos los individuos son válidos, el precio refleja directamente la calidad de la solución.
 - Es simple, interpretativa y fácil de usar en selección por ruleta.
-

5) Selección — ¿cómo elegimos padres?

Decisión: selección por **ruleta** (probabilidad proporcional al fitness).

Justificación:

- Los individuos con mayor precio tienen mayor probabilidad de reproducirse, pero los de menor fitness aún pueden participar (preserva diversidad).
- Fácil de implementar y didáctica para un TP.

Nota práctica: cuando la suma de fitnesses es muy desigual, puede llevar a convergencia prematura. En tal caso se puede usar *tournament selection* o escalado del fitness; pero para $n=10$ la ruleta suele ser suficiente.

6) Cruce (crossover) — cómo combinar padres para crear hijos

Decisión sugerida: **one-point crossover** (punto simple) o **uniform crossover** si querés más mezcla.

One-point (ejemplo): elegir un punto k entre 1 y $n-1$; hijo1 toma genes $0..k-1$ del padre A y $k..n-1$ del padre B; hijo2 hace lo inverso.

Justificación:

- One-point preserva bloques de solución que podrían ser útiles (epistasia baja en este problema, pero sigue siendo razonable).
 - Uniform mezcla más genes aleatoriamente y aumenta exploración.
-

7) Mutación — introducir variación aleatoria

Decisión: mutación bit a bit con probabilidad p_m pequeña (ej. $p_m = 0.02 \dots 0.05$). Al mutar se invierte el bit ($0 \leftrightarrow 1$).

Justificación:

- Evita estancamiento en óptimos locales; da pequeñas variaciones que la selección puede favorecer.
 - Debe ser baja para no destruir soluciones buenas.
-

8) Reparación / validación — asegurar la restricción de peso

Crucial: después de cruce y mutación **verificamos que el hijo cumple** $\text{peso} \leq C$. Si no, aplicamos reparación.

Estrategia de reparación recomendada (ratios): 1. Calcular la lista de ítems incluidos en el hijo.

2. Calcular $\text{ratio} = \text{precio/peso}$ para cada ítem incluido.

3. Ordenar los incluidos por ratio **ascendente** (peor ratio primero).

4. Eliminar (poner a 0) ítems de menor ratio uno a uno hasta que $\text{peso} \leq C$.

Justificación:

- Elimina primero los ítems que aportan menos precio por kg, conservando más valor total al reparar.
 - Es heurístico simple y efectivo. Mejor que eliminar aleatoriamente.
-

9) Reemplazo y elitismo — cómo formamos la nueva población

Decisión: formar una nueva población con los hijos; incluir **elitismo** (copiar los k mejores de la generación anterior, p. ej. $k=1$ o 2).

Justificación:

- Elitismo garantiza que la mejor solución no se pierda por cruces/mutaciones aleatorias.
 - Reemplazo completo con elitismo es estándar y simple.
-

10) Criterio de parada — cuándo terminamos

Opciones válidas (elige una o combina): - número máximo de generaciones G_{max} (p. ej. 200–500), o - no hay mejora en X generaciones seguidas, o - tiempo límite.

Justificación: fija un tope práctico y reproducible.

11) Análisis y verificación — comprobar resultados

Importante con $n=10$: podemos obtener la solución **óptima** por **fuerza bruta** ($2^{10} = 1024$ combinaciones) o por programación dinámica. Haz la fuerza bruta y **compara** con el mejor resultado del GA.

Justificación:

- Permite evaluar la calidad del GA (¿llega al óptimo? ¿qué tan cerca queda?) y justificar parámetros.

12) Parámetros iniciales recomendados y por qué

- **N** (población): 50 — equilibrio entre exploración y coste.
- **G_max**: 200-300 — suele ser suficiente para n pequeño; aumenta si la curva no converge.
- **pc** (prob. de cruce): 0.8-0.9 — favorece recombinación.
- **pm** (mutación por bit): 0.02-0.05 — pequeña pero útil.
- **elitismo_k**: 1 — protege mejor solución.

Ajusta si la convergencia es muy rápida (subir pm o N para más exploración).

13) Pseudocódigo (ordenado, paso a paso)

Inputs: precios[], pesos[], C, N, G_max, pc, pm, elitismo_k

1. Poblacion = inicializar N individuos válidos (cada uno: vector binario)
2. Evaluar fitness de cada individuo (precio si peso $\leq$ C)
3. Mejor_global = mejor individuo de Poblacion

```
for gen = 1..G_max:
    NuevaPob = []
    // Elitismo: copiar los k mejores individuos
    copiar elitismo_k mejores individuos de Poblacion a NuevaPob

    while len(NuevaPob) < N:
        padre1 = seleccionar_por_ruleta(Poblacion)
        padre2 = seleccionar_por_ruleta(Poblacion)
        hijo1, hijo2 = crossover_one_point(padre1, padre2, pc)
        hijo1 = mutar(hijo1, pm)
        hijo2 = mutar(hijo2, pm)
        hijo1 = reparar_si_excede(hijo1) // eliminar ítems por ratio hasta
cumplir C
        hijo2 = reparar_si_excede(hijo2)
        añadir hijo1, hijo2 a NuevaPob (cortar si supera N)

    Poblacion = NuevaPob
    evaluar fitness de Poblacion
    actualizar Mejor_global si hay mejora
    guardar mejor por generación (para gráficas)
```

Return Mejor_global (vector, precio, peso) y registro de mejor por generación

14) Ejemplo numérico paso a paso (mini-caso con 4 cajas)

Usamos las primeras 4 cajas del enunciado para ver la dinámica:

Datos (ítem i: precio, peso): 1. (100, 300) $\rightarrow$ ratio $100/300 = 0.333333...$ 2. (50, 200) $\rightarrow$ ratio $50/200 = 0.25$ 3. (115, 450) $\rightarrow$ ratio $115/450 \approx 0.255555...$ 4. (25, 145) $\rightarrow$ ratio $25/145 \approx 0.1724138$

Capacidad $C = 1000$.

Paso A — Inicialización (N = 4, ejemplo):

Generamos 4 individuos válidos (aleatorios controlados). Ejemplo: - I1 = $[1, 0, 1, 0]$ $\rightarrow$ precio = $100 + 115 = 215$; peso = $300 + 450 = 750$ (válido) - I2 = $[1, 1, 0, 0]$ $\rightarrow$ precio = $100 + 50 = 150$; peso = $300 + 200 = 500$ (válido) - I3 = $[0, 1, 1, 1]$ $\rightarrow$ precio = $50 + 115 + 25 = 190$; peso = $200 + 450 + 145 = 795$ (válido) - I4 = $[1, 1, 1, 1]$ $\rightarrow$ precio = $100 + 50 + 115 + 25 = 290$; peso = $300 + 200 + 450 + 145 = 1095$ (inválido)

Observación: I4 no es válido. Aplicamos reparación por ratio (eliminar ítems de menor ratio primero): - Ratios de incluidos: item4 (0.1724), item2 (0.25), item3 (0.2556), item1 (0.3333). - Eliminamos item4 (menor ratio): nuevo peso = $1095 - 145 = 950 \leq C$, nuevo precio = $290 - 25 = 265$.

- I4 reparado = $[1, 1, 1, 0]$ $\rightarrow$ precio **265**, peso **950**.

Estado inicial final (fitnesses): - I1: 215 - I2: 150 - I3: 190 - I4_repaired: 265

Paso B — Selección (ruleta) (ejemplo):

Suma fitness $S = 215 + 150 + 190 + 265 = 820$.

Probabilidades: - $P(I1) = 215/820 \approx 0.2622$ - $P(I2) = 150/820 \approx 0.1829$ - $P(I3) = 190/820 \approx 0.2317$ - $P(I4) = 265/820 \approx 0.3232$

Generamos dos padres por ruleta (ejemplo hipotético): supongamos salen $\text{padreA} = I4$ y $\text{padreB} = I1$.

Paso C — Cruce (one-point):

Elegimos punto de cruce $k = 2$ (entre gen2 y gen3, índice 0-based).

- padreA (I4) = $[1, 1, 1, 0]$ - padreB (I1) = $[1, 0, 1, 0]$ - hijo1 = primeros 2 genes de padreA + últimos 2 genes de padreB = $[1, 1 \mid 1, 0] \rightarrow [1, 1, 1, 0]$ (igual a padreA) - hijo2 = primeros 2 de padreB + últimos 2 de padreA = $[1, 0 \mid 1, 0] \rightarrow [1, 0, 1, 0]$ (igual a padreB)

En este ejemplo el cruce no produjo novedad; en otros casos sí.

Paso D — Mutación (pm pequeño)

Suponemos que en hijo2 el bit 2 (index 1) muta: hijo2 pasa `[1,0,1,0]` → `[1,1,1,0]` (se añade item2). Ese hijo tiene precio 265 y peso 950 (válido). Si la mutación hubiera activado un bit que provoca exceso, se repararía.

Paso E — Reparación (si necesario)

Si un hijo excede C se aplica la eliminación por ratio hasta cumplir.

Paso F — Reemplazo y siguiente generación

Incorporamos hijos (y copiamos el elitismo si aplica) y repetimos. Registramos el mejor de la generación (aquí I4_repaired con precio 265).

Con el tiempo, el algoritmo tenderá a favorecer combinaciones con mayor precio/peso total dentro del límite.

15) Por qué elegimos reparación por ratio y elitismo (justificación final)

- **Reparación por ratio:** conserva el máximo valor (precio) al quitar el “peor” por precio/kg. Es una heurística simple que mantiene soluciones de alta calidad tras cruces y mutaciones.
- **Elitismo:** protege la mejor solución encontrada de ser perdida por operadores estocásticos; mejora estabilidad y reproducibilidad del experimento.

16) Qué deberías implementar y en qué orden (checklist práctico)

1. Cargar datos (precios, pesos, C).
2. Implementar evaluación (precio y peso por individuo).
3. Implementar generación de individuos válidos (inicialización).
4. Implementar función `repair(ind)` basada en ratios.
5. Implementar selección por ruleta.
6. Implementar cruce (one-point) y mutación bit a bit.
7. Implementar ciclo GA con elitismo y registro de mejor por generación.
8. Implementar fuerza bruta para obtener el óptimo de referencia.
9. Ejecutar experimentos (variar N, pm, pc) y graficar la curva de mejor fitness por generación.
10. Documentar y comentar decisiones en el informe.

17) Consejos finales y errores comunes a evitar

- **No ignores la verificación de peso:** dejar individuos inválidos sin control distorsiona la búsqueda.

- **No poner mutación demasiado alta:** si pm es grande, el algoritmo se comporta como búsqueda aleatoria.
 - **Registrar semilla aleatoria** para poder reproducir resultados en el informe.
 - **Comparar con fuerza bruta** (n=10 permite esto) y reportar: mejor GA, mejor brute-force, y diferencias.
 - **Hacer múltiples corridas** (p.ej. 20 ejecuciones) y reportar media y desviación, no una sola corrida azarosa.
-

Si querés, ahora puedo: - Generar el **código Python completo** del GA con comentarios (listo para copiar), o
- Preparar una **demonstración paso a paso** con números reales (ejecutando el GA y mostrando la evolución),
o - Convertirlo a **C++** si lo necesitas en ese lenguaje.